VACHAN

A WhatsApp-native commitment agent for Indian small-business service teams
Built with Microsoft Agent Framework / Foundry-compatible code, running on a zero-Azure, near-zero-cost stack

1. The idea in one paragraph

Small service businesses (interior designers, event planners, contractors, CA/accounting firms, exporters) run their entire client-vendor-team coordination in WhatsApp chats and voice notes. Promises get made constantly ("I'll send the quote Friday", "tiles will arrive Monday", "payment by 5th") and then get lost in the scroll. **Vachan** (Hindi for 'promise') is an agent team that reads the conversations the owner forwards or shares with it, extracts every commitment (who, what, by when), follows up autonomously on the owner's behalf, escalates when something slips, and keeps an evidence-backed ledger of who kept their word. Over time it produces a **reliability score** for every vendor, client, and employee.

One-line pitch: 'Chat is where Indian business happens, but chat has no memory. Vachan gives it one, and acts on it.'

2. Why this is not a common project

- Existing meeting-note tools capture action items from *calls*. Existing CRMs need manual entry. Neither works on messy, multilingual WhatsApp threads with voice notes.
- The agent does not just summarize. It **acts**: sends nudges, collects confirmations, escalates, and reconciles promises against later messages ("delivered", photo proof, payment screenshot).
- The long-term asset is a **commitment graph** (who keeps promises, how late, on what kinds of tasks). That data does not exist anywhere today.

Honest caveat: I cannot guarantee nothing similar exists. Before you commit, spend one evening searching Product Hunt, the WhatsApp Business solution-provider directory, and YC's company list for 'commitment tracking', 'WhatsApp follow-up agent', and 'promise tracking'. Differentiation will come from the vertical focus and the reliability graph, not from the idea alone.

3. Wedge customer and problem

Start narrow: interior designers and home-renovation contractors in Bengaluru. A single project involves the client, 8-15 vendors (carpenter, tiles, electrician, painter), and the designer, all coordinated in WhatsApp groups. Delays and 'I never said that' disputes directly cost money and reputation.

Pain	What Vachan does
Promises buried in 400-message groups	Extracts commitments and builds a live 'who owes what by when' board
Owner spends hours chasing vendors	Agent sends polite, language-matched follow-ups (English / Hindi / Kannada) with owner approval rules
Disputes ('you never told me')	Evidence ledger: each commitment links to the original message, voice note timestamp, and proof photo
No data on which vendors are reliable	Vendor reliability score and delay patterns across projects

4. What the user experiences

- **Day 0:** Owner messages the Vachan WhatsApp number and forwards or exports a project chat. Within a minute they get a clean commitments summary.

- **Daily:** A 9 AM digest: 'Due today: 3. Overdue: 2. Waiting on you: 1.' Owner replies with a thumbs up or a voice note to approve follow-ups.
- **Autonomous follow-up:** Agent drafts 'Ramesh, tiles were promised Monday. Any update?' and sends it under rules the owner set (approve-first at the start, auto-send for low-risk nudges later).
- **Closure:** When a vendor replies 'delivered' or sends a photo, the agent verifies and closes the loop, or flags a mismatch.
- **Weekly:** Reliability report and a project-health summary the owner can show the client.

Voice notes matter: most Indian business chats are voice-heavy, so the pipeline must do speech-to-text on mixed-language audio before extraction.

5. Multi-agent design

Agent	Job	Notes
Ingest agent	Receives WhatsApp messages, voice notes, images, and exported chats; normalizes to a common format	Speech-to-text for voice; OCR/vision for photos
Commitment extractor	Finds promises, owners, deadlines, conditions; resolves 'Friday', 'next week', 'after payment'	Structured JSON output with confidence score
Reconciler	Matches later messages and proof to open commitments; closes, updates, or flags	Handles rescheduling and partial delivery
Follow-up agent	Drafts and sends nudges in the right language and tone, within owner-set rules	Human-in-the-loop approval tier by default
Escalation agent	Alerts the owner or notifies the client when a critical item slips	Severity rules per project
Ledger and scoring	Stores commitments, evidence links, and computes reliability scores	The long-term data moat
QA / eval agent	Samples outputs, checks extraction accuracy, blocks risky auto-sends	Your QA-architect strength, built in from day one

Orchestration: start with a single orchestrator that routes events (new message, timer fired, proof received) to these agents using Microsoft Agent Framework workflows. Persist state in Postgres so every agent run is replayable and auditable.

6. Tech stack (Foundry-compatible, no Azure account needed)

Key principle: Microsoft Foundry's Agent Service itself runs on Azure, so you cannot use the hosted service without a subscription. But the open-source **Microsoft Agent Framework** (the successor to Semantic Kernel and AutoGen) runs anywhere, and its model client is OpenAI-compatible. Build and test locally against free model endpoints, then change only the endpoint and key when you get Foundry access. No rewrite.

Layer	Choice
Agent framework	Microsoft Agent Framework (Python) with workflows and tool calling
Models (dev)	GitHub Models free tier, Google Gemini API free tier, Groq free tier, or local Ollama (Qwen / Llama / Gemma class models)
Speech-to-text	Whisper (faster-whisper) running locally or on a free GPU notebook; Gemini audio input as a fallback
Memory and storage	Postgres with pgvector (Supabase or Neon free tier)
Queue / timers	Postgres-based job table or Upstash Redis free tier; cron via GitHub Actions or the host's scheduler

Layer	Choice
API layer	FastAPI webhook service
Observability	Langfuse (self-hosted or free cloud tier) plus OpenTelemetry traces
Evals	promptfoo or DeepEval in GitHub Actions on a labeled chat dataset
Dashboard (later)	Next.js or Streamlit on a free host

7. Free infrastructure map

Free tiers change often. Treat the notes below as a starting checklist and confirm current limits on each provider's pricing page before you depend on them.

Need	Free options	Watch out for
Compute / hosting	Oracle Cloud Always Free (ARM VM, generous RAM); Google Cloud Run free tier; Render, Fly.io, or Railway starter tiers; Hugging Face Spaces for demos	Free web services may sleep when idle, which delays webhooks. Oracle signup can be picky about cards and regions.
Public webhook while developing	Cloudflare Tunnel (free) or ngrok free tier	URLs change on ngrok free; Cloudflare Tunnel is steadier.
LLM inference	GitHub Models (needs only a GitHub token); Gemini free tier; Groq free tier; OpenRouter free models; Ollama on your own machine	Rate limits and daily caps. Design for retries and model fallback.
Database	Supabase free (Postgres + pgvector), Neon free, or SQLite locally	Free projects can pause after inactivity.
Vector search	pgvector in the same Postgres; Qdrant Cloud free cluster	Keep one store at first to reduce moving parts.
Speech-to-text	faster-whisper locally; Google Colab or Kaggle free GPU for batch jobs	Mixed Hindi/Kannada/English accuracy needs testing on real clips.
CI and evals	GitHub Actions free minutes	Cache models and datasets to stay in limits.
Observability	Langfuse self-hosted via Docker, or its free cloud tier; Grafana Cloud free	Self-hosting needs RAM; the Oracle VM can handle it.
Domain and email	Cloudflare for DNS; free-tier transactional email for alerts	Business verification on WhatsApp needs a real business identity.
Azure itself (optional)	Azure free account credits or student/startup programs if you qualify later	Credits expire. Keep your code portable so you are never stuck.

8. WhatsApp integration: the reality check

This is the most important technical risk, so plan around it from day one.

Option	Cost	Fit
WhatsApp Business Cloud API (Meta)	A test number is free for development with a small list of recipients. Production use has per-message pricing; replies to a user within the 24-hour service window have historically been the cheapest or free.	The official, safe path. Required for a real product.
Telegram bot (prototype)	Free	Use it to build and test agent logic quickly before WhatsApp approval.
Twilio WhatsApp sandbox	Free sandbox for testing	Good for early demos.
Unofficial libraries (Baileys, whatsapp-web.js)	Free	**Not recommended.** They violate WhatsApp terms and risk number bans. Fine only for private experiments, never for customers.

Design decision that avoids the biggest limitation: the official API does not let a bot silently read an existing personal group. So Vachan's MVP works through three safe channels: (1) the owner **forwards** messages or voice notes to the Vachan number, (2) the owner uploads a WhatsApp **'Export chat'** file for bulk ingestion, and (3) the owner adds Vachan to a business-created group if Meta's Groups API is available to your account. Check the current Groups API eligibility in Meta's developer docs, because it has been rolling out gradually with restrictions. Also add clear consent messaging so group members know a bot is tracking commitments (this matters under India's DPDP Act).

9. Reliability, safety, and evaluation

- **Golden dataset:** collect 200-300 real (anonymized) chat snippets and hand-label commitments, owners, and deadlines. This becomes your regression suite.
- **Metrics:** commitment extraction precision and recall, deadline-resolution accuracy, false-nudge rate, owner approval rate, and time-to-close.
- **Autonomy ladder:** Level 0 summarize only, Level 1 draft and owner approves, Level 2 auto-send low-risk nudges, Level 3 auto-escalate. Move up only when metrics hold for a full week.
- **Guardrails:** never send money-related or accusatory messages automatically; quote the source message in every action; keep a full audit log.
- **Privacy:** encrypt stored chats, allow one-tap deletion per project, store only what is needed, and never train on customer data without explicit consent.

10. Six-week MVP plan

Week	Goal	Deliverable
1	Foundations: repo, FastAPI webhook, Telegram bot, Postgres schema, model client abstraction	Bot echoes and stores messages; model swappable by config
2	Commitment extractor on text plus the labeled dataset and first eval run	Baseline precision/recall report
3	Voice notes: Whisper pipeline, mixed-language tests; ingestion of exported chats	Voice-to-commitments working end to end
4	Reconciler and follow-up agent with approval flow; daily digest	Full loop on one real project (yours or a friend's)
5	Move to WhatsApp Cloud API test number; add Langfuse tracing and eval CI	Same loop running on WhatsApp
6	Pilot with 3-5 designers or contractors; collect feedback; build reliability score v1	Pilot report, demo video, and a one-page metrics sheet

11. The YC angle

Question YC asks	Answer to build toward
Who is the user and what is the pain?	Small service-business owners losing time and money to forgotten promises in chat
Why now?	Agents that can handle voice, multilingual text, and tool calls reliably are only now cheap enough; WhatsApp Business APIs are maturing
How does it make money?	Per-project or per-seat subscription (for example Rs 999-2,999/month per business); later, a paid vendor-verification network

Question YC asks	Answer to build toward
What is the moat?	Commitment graph and reliability scores across thousands of projects; workflow lock-in; language-specific accuracy tuning
How big can it get?	Starts in renovation and events, then expands to any business run on chat: logistics, exports, real-estate, agencies, field services
What proves traction?	Weekly active businesses, commitments closed per week, and measurable reduction in delayed tasks

12. Main risks and how to handle them

- **WhatsApp platform limits:** mitigate with forward/export flows and a Telegram fallback; keep a channel abstraction so you can add other channels.
- **Willingness to pay:** validate with 10 paid pilot conversations before building the dashboard; charge from the pilot stage.
- **Trust and privacy:** start at Level 1 autonomy, show the source message for every action, and make deletion easy.
- **Free-tier fragility:** abstract the model client, add fallbacks across providers, and cache aggressively.
- **Incumbents:** WhatsApp or CRMs may add summaries. Your defense is the vertical depth, action layer, and reliability graph.

13. Moving to Microsoft Foundry later

When you gain Azure access: deploy the same agents to Foundry Agent Service, swap the free model endpoint for a Foundry-hosted model deployment, move tracing to Azure Monitor if you want it, and keep Postgres and the WhatsApp layer unchanged. Because everything is behind a model-client interface and standard OpenTelemetry, the migration should be configuration, not a rewrite. Foundry features and names evolve quickly, so check Microsoft's current docs when you make the move.

14. Your first three actions this week

- Spend one evening on the competitor check described in section 2.
- Interview 5 interior designers or contractors and ask to see one real project chat (anonymized) to start your labeled dataset.
- Stand up the Telegram + FastAPI + Postgres skeleton with the model client abstraction, and run the extractor on 20 real snippets.

Note: free-tier limits, WhatsApp policies, and Foundry features change frequently. Verify each against the provider's current documentation before relying on it.